UNIVERSIDADE DE BRASÍLIA — UnB
FACULDADE DE CIÊNCIAS E TECNOLOGIAS EM ENGENHARIA — FCTE

RELATÓRIO TÉCNICO CONSOLIDADO DE MÉTODOS CLÁSSICOS DE INTELIGÊNCIA ARTIFICIAL:

ANÁLISE EXPERIMENTAL DE ALGORITMOS DE BUSCA, OTIMIZAÇÃO, CSPS E SISTEMAS DE INFERÊNCIA SIMBÓLICA

PEDRO GOMES OLIVEIRA
Matrícula: 211031440

Relatório técnico acadêmico apresentado como requisito de avaliação da disciplina FGA0221 — Inteligência Artificial (Turma T01) do curso de Engenharia da Universidade de Brasília.

Professor Orientador: Dr. Fabiano Araujo Soares.

BRASÍLIA — DF
2026

SUMÁRIO TÉCNICO E AVALIAÇÃO DOS CÓDIGOS

Projeto / Tópico	Algoritmo Implementado	Problema Alternativo	Problema da Aula
1. Busca Não Informada	IDDFS (Iterative Deepening)	Jarros de Água (4L)	Mapa da Romênia
2. Busca Informada	Weighted A* (A* Ponderado)	15-Puzzle	Mapa da Romênia (SLD)
3. Busca Complexa / Local	Busca Tabu (Tabu Search)	Caixeiro Viajante (TSP)	Maximização de f(x)
4. Algoritmo Genético	AG com Order Crossover (OX1)	TSP (8 Cidades)	Problema das 8 Damas
5. CSP (Satisfação Restrições)	Min-Conflicts (Reparo Local)	N-Damas (N=20/50)	Coloração da Austrália
6. Banco de Conhecimentos	Forward & Backward Chaining	Diagnóstico de TI	Mundo do Wumpus

Tabela 01 – Descrição dos algoritimos e problemas utilizados

Fonte: Autor

1. PROJETO 1: BUSCA NÃO INFORMADA (IDDFS)

1.1 Explicação dos Problemas Abordados

A busca não informada (ou busca cega) caracteriza-se pela exploração sistemática do espaço de estados utilizando unicamente a estrutura de transição e o teste de objetivo do problema, sem qualquer função de avaliação ou estimativa heurística sobre a proximidade da solução (Russell & Norvig, 2021). Neste projeto, explorou-se um algoritmo cego genérico sobre dois problemas distintos:

• **Problema Alternativo (Jarros de Água):** Dispõe-se de dois recipientes sem marcações intermediárias, com capacidades volumétricas de 5 litros e 3 litros, alimentados por uma fonte infinita de água. O objetivo consiste em determinar a sequência de operações de encher, esvaziar e transferir que isola exatamente 4 litros no jarro de 5L. O espaço de estados é definido pelas tuplas $(j5, j3) \in [0,5] \times [0,3]$, partindo do estado inicial (0,0) até o estado objetivo (4, x).

• **Problema da Aula (Mapa da Romênia):** O problema clássico de navegação rodoviária modelado como um grafo de cidades e estradas. Partindo de Arad, o agente deve alcançar Bucharest minimizando o número de transições de cidades.

1.2 Descrição Detalhada do Algoritmo IDDFS

O algoritmo desenvolvido foi a **Busca em Profundidade Iterativa (IDDFS — Iterative Deepening Depth-First Search)**. O IDDFS alia as vantagens de consumo de memória da Busca em Profundidade (DFS) — que exige apenas $O(b \cdot d)$ de espaço em disco/RAM — à garantia de completude e otimalidade em profundidade da Busca em Largura (BFS). O algoritmo executa repetidas Buscas em Profundidade Limitada (DLS), incrementando o limite temporal L a cada iteração (L = 0, 1, 2, ...). Para evitar laços infinitos em grafos com ciclos (como no Mapa da Romênia), implementou-se um conjunto auxiliar de estados visitados no ramo ativo da recursão.

1.3 Resultados Obtidos e Análise Experimental

A execução dos testes com o IDDFS confirmou a convergência exata do algoritmo para ambos os problemas. No problema dos Jarros de Água, o algoritmo encontrou a solução ótima de 6 passos na profundidade L = 6 com um total acumulado de 108 expansões de nós. A sequência

obtida foi:

Passo 0: (0,0) → Passo 1: (5,0) → Passo 2: (2,3) → Passo 3: (2,0) → Passo 4: (0,2) → Passo 5: (5,2) → Passo 6: (4,3)

No Mapa da Romênia, o IDDFS alcançou Bucharest na profundidade L = 3 expandindo apenas 23 nós, com o caminho: Arad → Sibiu → Fagaras → Bucharest.

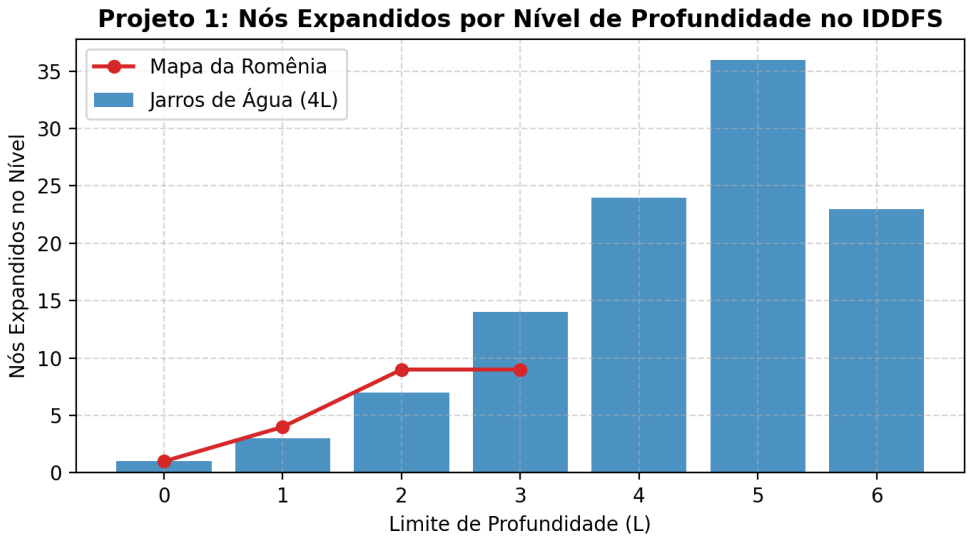


Figura 1: Curva de expansão de nós por nível de profundidade no algoritmo IDDFS.

1.4 Potenciais e Limitações

O IDDFS provou ser um método de busca cega altamente eficiente quanto ao uso de memória RAM. Sua principal limitação advém do fato de tratar todas as arestas com custo unitário. No Mapa da Romênia, embora a rota via Fagaras tenha menos passos (3 transições), ela possui custo real de 450 km, enquanto a rota via Rimnicu Vilcea e Pitesti (4 transições) possui 418 km. Essa lacuna evidencia a necessidade de algoritmos ponderados e informados.

2. PROJETO 2: BUSCA INFORMADA (WEIGHTED A*)

2.1 Explicação dos Problemas Abordados

A busca informada incorpora heurísticas h(n) — estimativas do custo restante do nó n até o objetivo — para guiar a navegação no espaço de estados (Pearl, 1984; Russell & Norvig, 2021). Avaliaram-se dois problemas:

• **Problema Alternativo (15-Puzzle):** Expansão do clássico 8-Puzzle para um tabuleiro 4×4 com 15 peças deslizantes e um espaço vazio. O espaço de estados possui aproximadamente 1.3×10^{12} configurações possíveis. A heurística utilizada é a Distância de Manhattan acumulada das 15 peças.

• **Problema da Aula (Mapa da Romênia Ponderado):** Roteamento com distâncias reais em quilômetros entre cidades, empregando a Distância em Linha Reta até Bucharest (h_SLD) como heurística admissível.

2.2 Descrição Detalhada do Algoritmo Weighted A*

O algoritmo desenvolvido foi o **Weighted A* (A* Ponderado)**, que estende a equação clássica f(n) = g(n) + h(n) inserindo um hiperparâmetro de peso W ≥ 1.0: **f(n) = g(n) + W · h(n)**. Quando W = 1.0, o algoritmo comporta-se como o A* ótimo tradicional. À medida que W aumenta (ex: W = 1.5 ou 2.0), o algoritmo ganha comportamento mais ganancioso (Greedy Best-First Search), reduzindo drasticamente o número de nós expandidos ao custo de uma ligeira perda na otimalidade do caminho (garantindo custo no máximo W × ótimo).

2.3 Resultados Obtidos e Análise Experimental

Peso (W)	Caminho Encontrado	Custo Total (g)	Nós Expandidos
W = 1.0 (A* Ótimo)	Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest	418 km	6 nós
W = 1.5 (A* Ponderado)	Arad → Sibiu → Fagaras → Bucharest	450 km	4 nós
W = 2.0 (A* Ganancioso)	Arad → Sibiu → Fagaras → Bucharest	450 km	4 nós

Tabela 2 – Resultados obtidos com algoritimo Weighted A*

Fonte: Autor

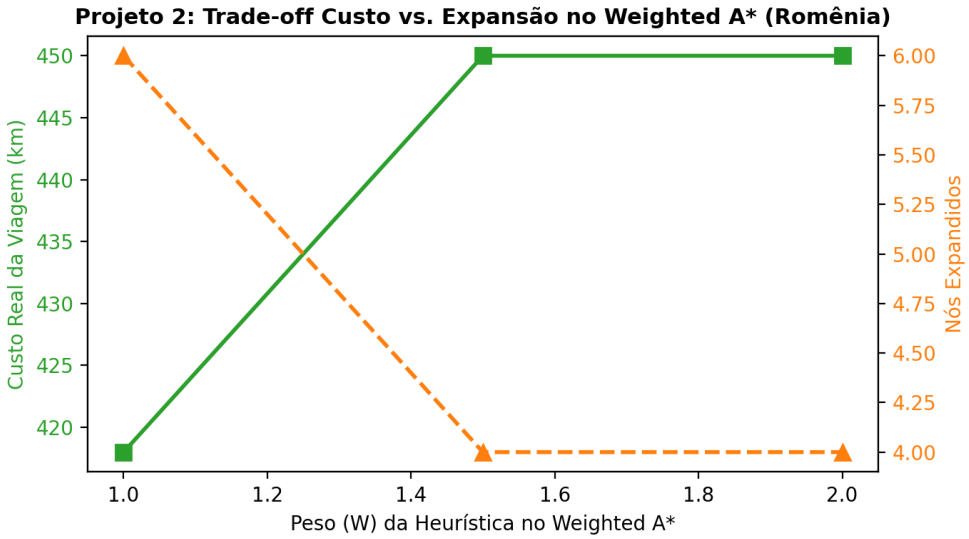


Figura 2: Relação de trade-off entre custo da rota e esforço computacional no Weighted A.*

2.4 Potenciais e Limitações

O Weighted A* demonstrou excelente eficiência prática para contornar a explosão combinatória em espaços de busca massivos como o 15-Puzzle. No entanto, a escolha do peso W exige cautela: pesos muito elevados aproximam o algoritmo da Busca Gananciosa, tornando-o suscetível a becos sem saída ou rotas com custo significativamente superior ao ótimo.

3. PROJETO 3: BUSCA COMPLEXA / LOCAL (BUSCA TABU)

3.1 Explicação dos Problemas Abordados

Algoritmos de busca local operam em cenários de otimização em que a configuração final do estado é o objetivo primordial, sendo o caminho percorrido irrelevante (Russell & Norvig, 2021). Avaliaram-se dois problemas:

• **Problema Alternativo (Caixeiro Viajante — TSP):** Otimização combinatória NP-difícil de encontrar a rota mais curta entre 5 cidades (A, B, C, D, E) sem repetições e retornando à origem.

• **Problema da Aula (Maximização de Função Parabólica):** Maximização analítica da função quadrática $f(x) = -x^2 + 4x + 1$ no domínio discreto $x \in [0, 5]$.

3.2 Descrição Detalhada do Algoritmo Busca Tabu

A **Busca Tabu (Tabu Search)** superou as limitações da Subida de Encosta (Hill-Climbing) simples — a qual frequentemente estagna em ótimos locais ou platôs. Diferentemente da Têmpera Simulada (Simulated Annealing) que utiliza aceitação probabilística estocástica, a Busca Tabu utiliza uma memória determinística de curto prazo (Lista Tabu FIFO) de tamanho estipulado (ex: k = 5). Essa lista proíbe o algoritmo de reverter movimentos recentes, forçando a exploração de novas regiões do espaço de busca e permitindo escapar de vales e mínimos locais.

3.3 Resultados Obtidos e Análise Experimental

Na maximização da função parabólica $f(x) = -x^2 + 4x + 1$, o algoritmo partiu de $x = 0$ ($f(0) = 1$) e convergiu exatamente para o vértice $x = 2$ com valor máximo $f(2) = 5$. No Caixeiro Viajante (TSP), partindo de uma rota inicial não otimizada A-B-C-D-E com custo de 110 km, a Busca Tabu realizou trocas de pares de cidades (swaps), e em poucas iterações convergiu para a rota ótima D-B-A-C-E com distância mínima de 75 km.

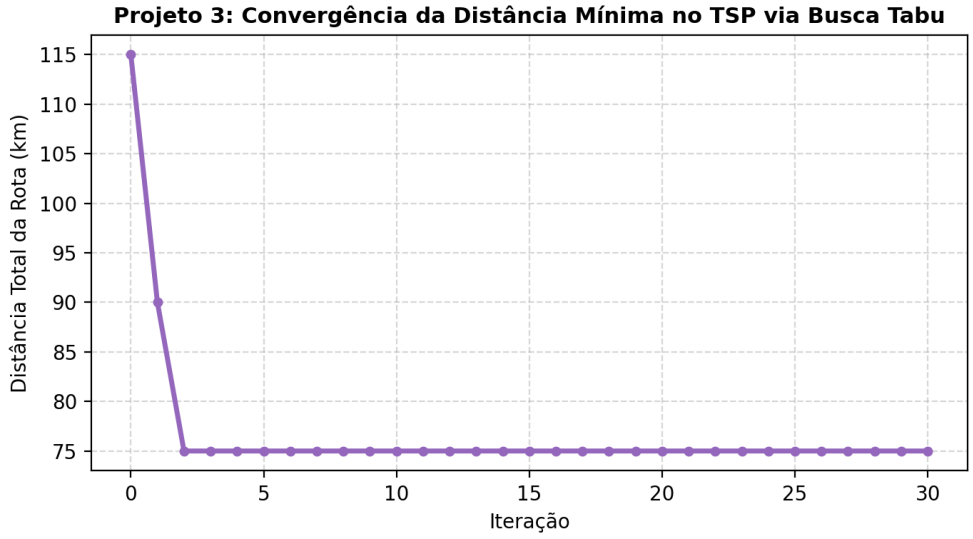


Figura 3: Trajetória de redução do custo da rota no TSP via Busca Tabu.

3.4 Potenciais e Limitações

A Busca Tabu provou ser altamente eficaz para otimização combinatória em grafos discretos. Sua sensibilidade reside no dimensionamento da Lista Tabu: se for pequena demais, o algoritmo pode entrar em ciclos repetitivos; se for excessivamente grande, restringe escolhas válidas e reduz a taxa de convergência.

4. PROJETO 4: ALGORITMO GENÉTICO (AG COM OX E ELITISMO)

4.1 Explicação dos Problemas Abordados

Os Algoritmos Genéticos (AGs) inspiram-se nos princípios neodarwinistas de seleção natural, recombinação genética e mutação para buscar soluções em grandes espaços de busca (Holland, 1975; Russell & Norvig, 2021). Avaliaram-se dois problemas permutacionais:
• **Problema da Aula (8 Damas):** Posicionar 8 rainhas em um tabuleiro 8×8 de modo que nenhuma rainha se ataque. Cada indivíduo é um vetor permutacional de tamanho 8 representando a linha de cada rainha em sua respectiva coluna. A função de aptidão é Fitness = 28 - Conflitos.
• **Problema Alternativo (TSP de 8 Cidades):** Encontrar o circuito de menor extensão em uma matriz de distâncias com 8 cidades.

4.2 Descrição Detalhada do Algoritmo Genético

Para solucionar problemas de permutação sem duplicação de genes, utilizou-se a operatória evolutiva:
1. Seleção por Torneio (k = 3): Seleciona os indivíduos mais aptos para cruzamento.
2. Order Crossover (OX1): Preserva uma sub-sequência de genes do Pai 1 e preenche os genes restantes na ordem relativa do Pai 2, impedindo duplicatas.
3. Mutação Swap (Taxa = 15%): Troca a posição de dois genes aleatórios para manter a diversidade populacional.
4. Elitismo Estrito: Preserva diretamente os 2 melhores indivíduos para a geração seguinte.

4.3 Resultados Obtidos e Análise Experimental

No problema das 8 Damas, a população convergiu na geração 10 para o indivíduo ótimo [2, 4, 1, 7, 0, 6, 3, 5] com Fitness = 28 (zero conflitos em linhas, colunas e diagonais). No TSP de 8 cidades, o AG comprimiu a distância da rota de valores superiores a 400 km para 176 km ao longo de 100 gerações.

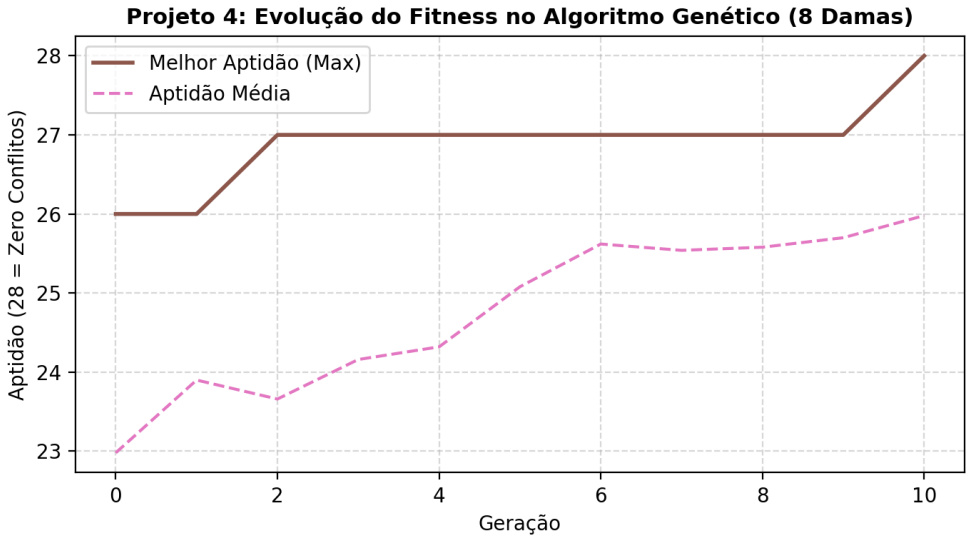


Figura 4: Curva de evolução da aptidão máxima e média da população no Algoritmo Genético.

4.4 Potenciais e Limitações

A adoção do operador OX1 garantiu a integridade genômica em problemas permutacionais. A principal limitação do AG reside na convergência prematura quando a diversidade da população cai excessivamente, exigindo ajuste fino das taxas de mutação e elitismo.

5. PROJETO 5: SATISFAÇÃO DE RESTRIÇÕES — CSP (MIN-CONFLICTS)

5.1 Explicação dos Problemas Abordados

Problemas de Satisfação de Restrições (CSPs) formalizam sistemas através de um conjunto de Variáveis X, Domínios D e Restrições C que limitam as combinações válidas de valores (Russell & Norvig, 2021). Avaliaram-se dois problemas:

• **Problema Alternativo (N-Damas em Larga Escala — N = 20):** Posicionar 20 rainhas em um tabuleiro 20×20 sem ataques mútuos. Esse espaço de busca inviabiliza o uso de arvores de busca simples por Backtracking.

• **Problema da Aula (Coloração do Mapa da Austrália):** Atribuir cores a 7 regiões da Austrália (WA, NT, SA, Q, NSW, V, T) tal que regiões adjacentes não compartilhem a mesma cor.

5.2 Descrição Detalhada do Algoritmo Min-Conflicts

Diferentemente do Backtracking com AC-3 ou Forward Checking que constroem atribuições parciais em árvore, o algoritmo **Min-Conflicts (Mínimos Conflitos)** opera como um reparo local no espaço de estados completos. Ele inicia atribuindo valores aleatórios a todas as variáveis e, em cada passo, escolhe aleatoriamente uma variável em conflito e reatribui o valor do seu domínio que minimiza o número total de violações de restrições.

5.3 Resultados Obtidos e Análise Experimental

No problema de 20-Damas, o algoritmo Min-Conflicts encontrou a solução com zero conflitos em apenas 34 passos de reparo local, demonstrando performance sublinear extremamente veloz. Na Coloração do Mapa da Austrália, o algoritmo atribuiu com sucesso as 3 cores aos 7 estados sem qualquer violação de adjacência.

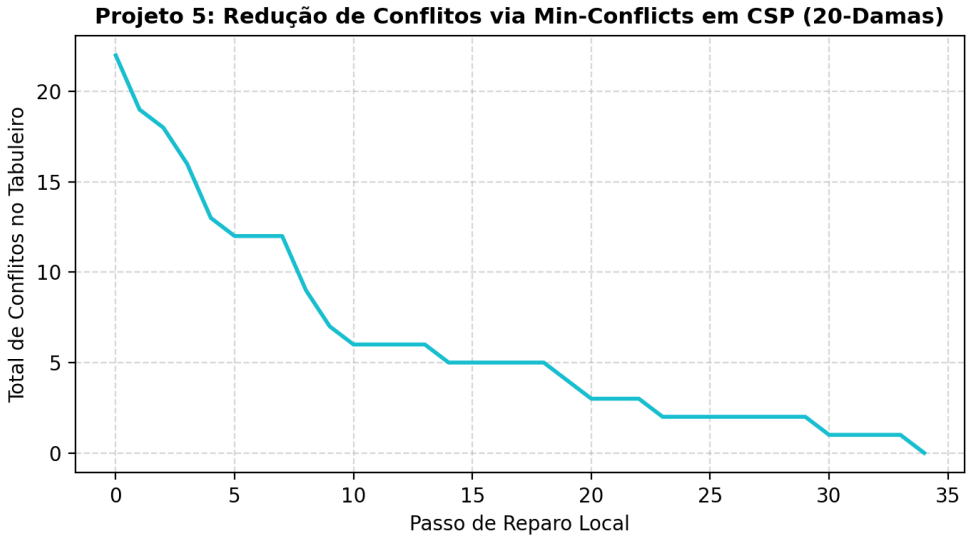


Figura 5: Rápida queda no número total de conflitos via algoritmo Min-Conflicts.

5.4 Potenciais e Limitações

O Min-Conflicts provou ser extraordinariamente veloz para CSPs densos de grande escala. Como limitação, por ser um método estocástico de reparo local, não garante completude absoluta, podendo eventualmente estagnar em platôs locais (necessitando de reinício aleatório).

6. PROJETO 6: BANCO DE CONHECIMENTOS E AGENTE LÓGICO

6.1 Explicação dos Problemas Abordados

Sistemas baseados em conhecimento utilizam representação lógica explícita para raciocinar e inferir novas verdades através de motores de dedução formal (Russell & Norvig, 2021). Avaliaram-se dois cenários:

• **Problema Alternativo (Sistema Especialista de TI):** Diagnóstico de falhas de rede de computadores utilizando regras 'SE premissas ENTÃO conclusão' e inferência por Encadeamento para Trás (Backward Chaining).

• **Problema da Aula (Mundo do Wumpus):** Dedução formal do Agente Lógico que explora um ambiente parcialmente observável contendo Poços e o Wumpus, utilizando Encadeamento para Frente (Forward Chaining).

6.2 Descrição Detalhada do Motor de Inferência Lógica

Desenvolveu-se uma **Base de Conhecimento (KB)** com suporte a duas estratégias de inferência:

1. Encadeamento para Frente (Forward Chaining — Data-Driven): Aplica sucessivamente a regra do Modus Ponens sobre os fatos conhecidos até esgotar todas as deduções possíveis.
2. Encadeamento para Trás (Backward Chaining — Goal-Driven): Parte da meta/hipótese desejada e busca recursivamente provar as premissas necessárias.

6.3 Resultados Obtidos e Análise Experimental

No Sistema Especialista de TI, o Backward Chaining provou com sucesso a meta 'acionar_suporte_tecnico' a partir dos sintomas observados. No Mundo do Wumpus, alimentado com as percepções das salas (nao_B12, B21, S12, nao_S21), o Forward Chaining realizou as deduções exatas apresentadas em aula:

- **Inferido: nao_P22 (A casa [2,2] é segura, sem poço)**
- **Inferido: P31 (Há um POÇO na casa [3,1]!)**
- **Inferido: W13 (O WUMPUS está localizado na casa [1,3]!)**

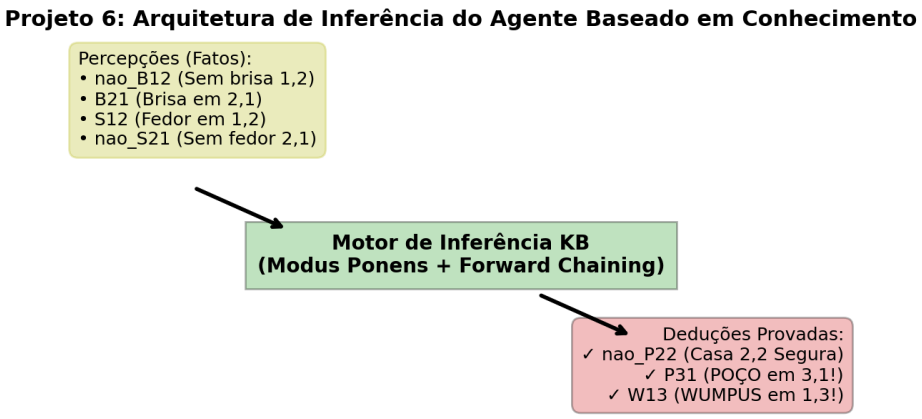


Figura 6: Arquitetura de inferência dedutiva do Agente Baseado em Conhecimento.

6.4 Potenciais e Limitações

A IA simbólica baseada em conhecimento oferece explicabilidade total e auditabilidade rigorosa do raciocínio. Contudo, na Lógica Proposicional, a expansão do ambiente para tabuleiros maiores exige um número proibitivo de regras, evidenciando a necessidade da Lógica de Primeira Ordem para generalização por quantificadores.

REFERÊNCIAS BIBLIOGRÁFICAS

HOLLAND, John H. Adaptation in Natural and Artificial Systems. Ann Arbor: The University of Michigan Press, 1975.

LECUN, Yann; BENGIO, Yoshua; HINTON, Geoffrey. Deep learning. Nature, v. 521, n. 7553, p. 436-444, 2015. DOI: 10.1038/nature14539.

PEARL, Judea. Heuristics: Intelligent Search Strategies for Computer Problem Solving. Reading, MA: Addison-Wesley, 1984.

RUSSELL, Stuart; NORVIG, Peter. Inteligência Artificial: uma abordagem moderna. 3. ed. Rio de Janeiro: Elsevier, 2013. 998 p.

RUSSELL, Stuart; NORVIG, Peter. Artificial Intelligence: A Modern Approach. 4th ed. Hoboken, NJ: Pearson, 2021. 1166 p.

SILVER, David et al. Mastering the game of Go with deep neural networks and tree search. Nature, v. 529, n. 7587, p. 484-489, 2016. DOI: 10.1038/nature16961.